MINI PROJECT — INFRA NETWORK ENGINEER · 우상목

데이터센터 네트워크 제안서.

Kubernetes 클러스터(컨트롤러 3대, 워커 5대)와 Ceph 스토리지 4대를 랙 3개에 넣는 네트워크를 설계했습니다. 장비 목록부터 늘어놓기보다 데이터가 실제로 어떤 길로 다니는지를 먼저 보여 드리고, 왜 그렇게 만들었는지를 설명하는 순서로 정리했습니다.

지원자 우상목 · 2026년 10월 · **첨부 코드** elice-infra/ (설계 파일, 설정 생성기, 시험 환경, Ansible)

요약

SUMMARY

이 설계에서 가장 먼저 정한 원칙은 트래픽 종류마다 길을 따로 두는 것입니다. 손님의 웹 요청, 서버가 스토리지에 읽고 쓰는 데이터, 스토리지끼리의 복제, 관리자의 접속은 양도 다르고 문제가 생겼을 때의 영향도 다릅니다. 한 길에 섞어 두면 복제가 몰릴 때 서비스까지 느려지고, 한 곳이 뚫리면 다른 곳도 위험해집니다. 그래서 트래픽마다 연결과 네트워크를 나눴고, 서로 다른 구역 사이는 방화벽을 통해서만 오가게 했습니다.

설정은 손으로 만들지 않았습니다. 랙, 장비, 주소 같은 설계 값을 파일 하나(YAML)에 적고, 스위치 설정과 IP 표, 배선표는 모두 이 파일에서 자동으로 만들었습니다.

PART 1

데이터가 다니는 길

먼저 전체 구조를 보고, 손님의 요청부터 장애가 났을 때의 우회까지 데이터가 실제로 지나가는 길을 따라갑니다.

PART 2

구축 계획

그 길을 까는 데 필요한 장비와 배선, 서버 연결, 구역 나누기, 주소, 대역폭을 정리합니다.

PART 3

운영 계획

그 길이 계속 잘 유지되도록 감시하고, 자동화하고, 고장에 대응하고, 늘려 가는 방법입니다.

PART 1

데이터가 다니는 길

전체 구조를 보고, 데이터가 실제로 지나가는 길을 따라갑니다

1-1

전체 구조

TOPOLOGY

서버 12대를 랙 3개에 나눴습니다. 랙마다 서버가 연결되는 스위치(리프) 2대를 두고, 모든 리프를 위쪽 스위치(스파인) 2대에 연결했습니다. 이런 구조를 스파인-리프라고 하며, 어느 서버에서 어느 서버로 가든 **리프 → 스파인 → 리프** 세 칸이면 도착합니다.

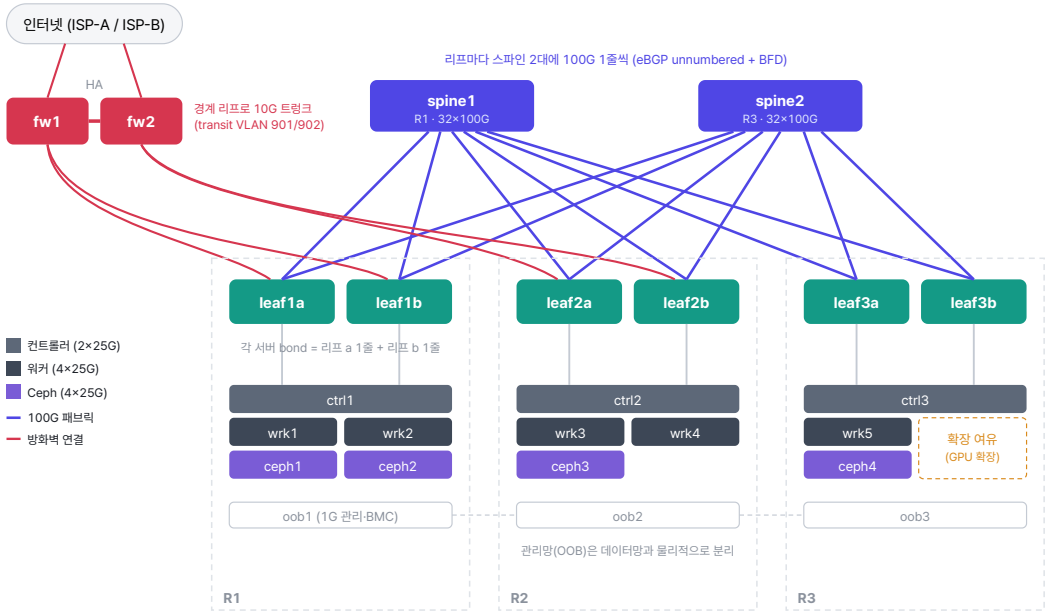


그림 1. 전체 구조 (스위치 11대, 방화벽 2대, 서버 12대). 모든 서버는 같은 랙의 리프 두 대에 한 줄씩 연결된다

장비	수량·위치	하는 일
스파인	2대 (R1, R3)	모든 리프를 잇는 중심. 한 대가 꺼져도 나머지 한 대가 받습니다.
리프	6대 (랙마다 2대)	서버가 꽂히는 랙 맨 위 스위치. R1·R2의 리프는 방화벽도 연결합니다.
서버	12대	모든 연결을 같은 랙의 리프 두 대에 한 줄씩 꽂아, 리프 하나가 꺼져도 끊기지 않습니다.
방화벽	2대 (R1, R2)	인터넷과 내부, 내부 구역 사이를 오가는 유일한 문. 두 대가 한 쌍으로 서로를 대신합니다.
관리망 스위치	3대 (랙마다 1대)	서비스용 네트워크와 따로, 장비를 관리하는 연결만 모읍니다.

스파인과 방화벽, 컨트롤러는 일부러 서로 다른 랙에 나눠 뒀습니다. 랙 하나의 전원이 나가도 나머지 랙에서 서비스가 이어지게 하기 위해서입니다.

1-2

트래픽 네 종류와 길 나누기

FLOW MAP

이 네트워크에는 성격이 다른 트래픽 네 종류가 다닙니다. 아래 그림처럼 한 장에 겹쳐 보면, 서로 다른 트래픽이 같은 길을 다투지 않게 나뉘어 있습니다. 양이 많은 스토리지와 복제 트래픽은 방화벽을 거치지 않고 각자의 연결로 다니고, 방화벽은 구역을 넘나드는 트래픽(손님 요청, 관리 접속)만 지납니다.

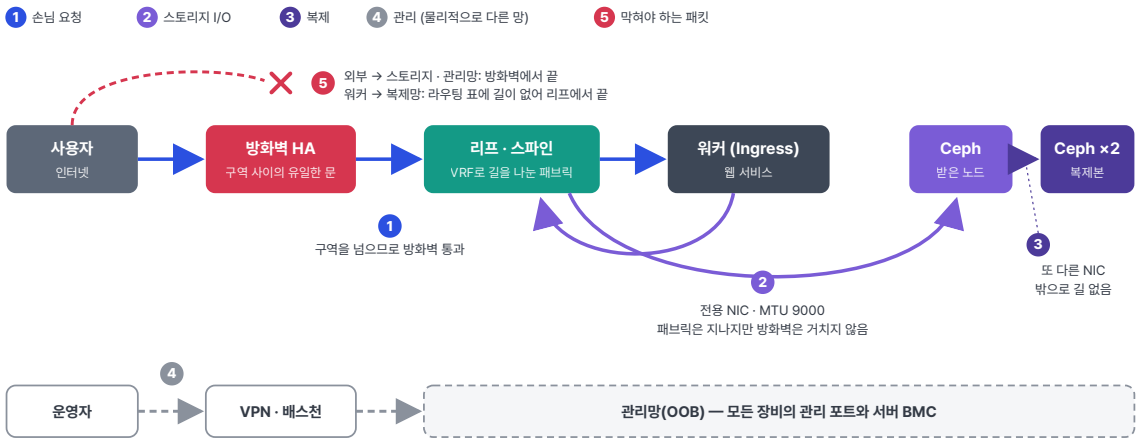


그림 2. 트래픽 지도. 번호는 아래 1-3부터의 순서와 같다

트래픽	어디서 어디로	그래서 이렇게 나눴다
손님 요청	인터넷 → 방화벽 → 워커	밖에서 들어오므로 방화벽을 꼭 거치게 함
스토리지 읽기·쓰기	워커 ↔ Ceph	양이 많아 전용 연결을 따로 둠. 방화벽은 안 거침
스토리지 복제	Ceph ↔ Ceph	장애 때 폭주하므로 또 다른 연결로 분리. 밖으로 나가는 길 없음
관리 접속	관리자 → 장비	서비스망이 멈춰도 쓸 수 있게 관리망을 따로 둠

네트워크는 VRF(같은 스위치 안에서 라우팅 표를 아예 따로 쓰는 칸막이) 세 개로 나눴습니다. 서비스용(PROD), 스토리지용(STORAGE), 복제용(REPL)입니다. 칸막이가 다르면 같은 스위치를 지나도 서로의 길을 모릅니다.

1-3

손님의 요청이 웹 서비스에 닿기까지

CLIENT

사용자가 브라우저에서 엘리스 서비스를 열면, 요청은 공인 IP의 443번 포트로 들어옵니다. 그림의 주황색 번호를 따라가 보겠습니다.

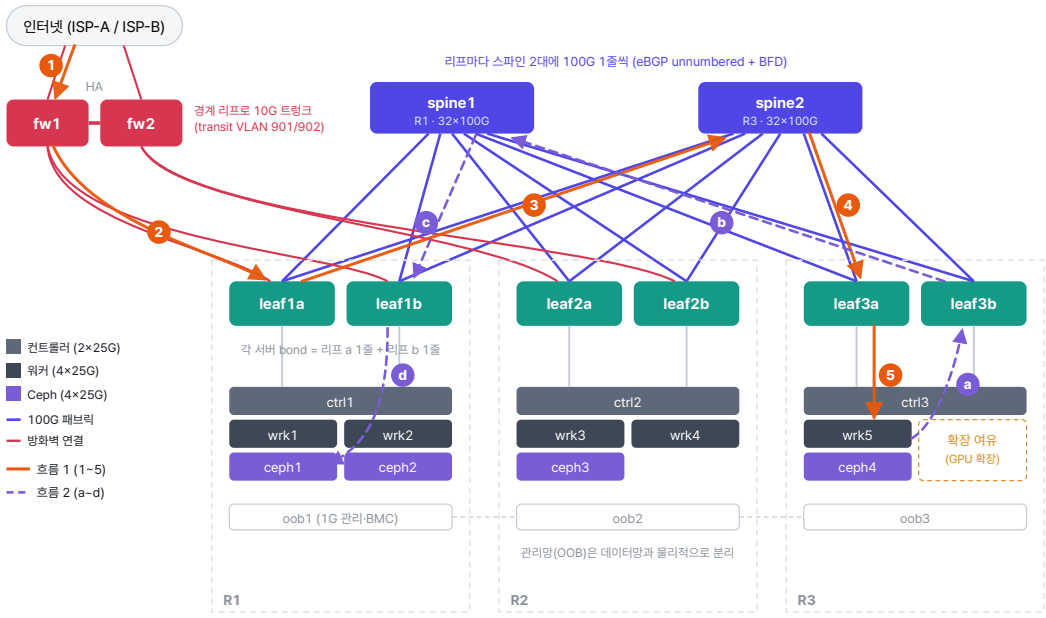


그림 3. 손님 요청(주황 번호)과 스토리지 읽기(보라 알파벳)가 실제로 지나가는 길

#	구간	무슨 일이	왜
1	인터넷 → 방화벽	열어 둔 포트(80/443)인지 확인하고, 공인 주소를 내부 서비스 주소로 바꿔 넘김	밖에 열린 문은 주소 하나, 포트 두 개뿐. 회선 2개, 방화벽 2대라 하나가 죽어도 문은 열려 있음
2	방화벽 → 리프	서비스 주소가 어느 워커에 있는지 찾고, 다른 랙으로 보내기 위해 한 번 포장(VXLAN)함	서비스가 떠 있는 워커들이 자기 위치를 알려 주므로, 여러 워커에 손님을 나눠 보낼 수 있음
3	리프 → 스파인	포장 겉면의 목적지만 보고 전달. 스파인 두 대 중 하나로 나눠 보냄	스파인은 단순한 고속도로. 한 대가 죽어도 다른 쪽으로 감
4	스파인 → 리프	포장을 풀고 서비스 칸막이(PROD) 안에서 목적지 워커를 찾음	칸막이가 달라 스토리지·복제 트래픽과 섞이지 않음
5	리프 → 워커	서비스용 연결로 도착해 웹 서비스가 받음	서비스 연결은 인터넷과 같은 크기(1500바이트)로 맞춰 큰 패킷이 버려지지 않게 함

응답은 같은 길을 거꾸로 갑니다. 방화벽은 들어온 연결을 기억하고 있어서, 나가는 응답을 다시 공인 주소로 바꿔 줍니다.

VXLAN: 랙이 달라도 같은 네트워크에 있는 것처럼 쓰려고, 원래 패킷을 한 번 더 감싸 보내는 방식입니다. 택배 상자를 큰 상자에 넣어 보내고 도착지에서 꺼내는 것과 비슷합니다.

1-4

워커가 스토리지에서 데이터를 읽기까지

STORAGE

웹 서비스가 사용자 데이터를 읽어야 합니다. 데이터는 네트워크 너머의 Ceph에 있습니다. R3의 wrk5가 R1의 ceph1에게 요청을 보냅니다. 그림 3의 보라색 알파벳 길입니다.

#	구간	무슨 일이	왜
a	wrk5 → 리프	서비스용이 아닌 스토리지 전용 연결로 나감	손님 트래픽과 같은 줄을 다투지 않고, 방화벽도 거치지 않아 속도가 묶이지 않음
b	리프 → 스파인	큰 패킷(9000바이트) 그대로 포장해 R1으로 보냄	스토리지는 한 번에 크게 보내야 효율이 좋음. 스위치 사이는 포장해도 들어가는 크기(9216)로 맞춤
c	스파인 → 리프	ceph1이 연결된 리프 두 대 중 한쪽으로	리프 하나가 죽어도 다른 쪽으로 감
d	리프 → ceph1	포장을 풀어 ceph1에 전달, 같은 길로 응답	랙이 달라도 같은 네트워크처럼 동작

패킷 크기 설정이 한 곳이라도 어긋나면 작은 ping은 통과하는데 큰 데이터만 사라지는 장애가 생깁니다. 그래서 크기를 구간마다 정해 두고 따로 감시합니다(3-2). 실제로 막히기 쉬운 곳은 스파인이 아니라, 여러 Ceph가 한 워커로 동시에 응답하는 마지막 구간입니다. 이 구간은 스토리지 트래픽에 대역을 우선 보장해 다룹니다(2-5).

1-5

데이터를 쓰면 일어나는 복제

REPLICATION

사용자가 과제를 제출해 wrk5가 1MB를 씁니다. 데이터는 ceph1에 도착하지만 여기서 끝나지 않습니다.

#	구간	무슨 일이	왜
1	wrk5 → ceph1	1-4와 같은 길로 1MB 도착	-
2	ceph1 → 다른 랙의 Ceph 2대	복제 전용 연결로 같은 데이터를 두 곳에 더 저장 → 2MB가 더 흐름	쓰기 1이 복제 2를 부름. 손님 쪽 연결과 섞이면 서비스가 느려지므로 연결부터 분리

3	→ wrk5	세 곳 모두 저장했다는 확인이 모여야 완료 응답	복제가 느리면 손님이 느낌. 그래서 복제 연결도 넉넉하게(25G 두 줄)

이 길이 가장 붐비는 때는 Ceph 한 대가 고장 났을 때입니다. 남은 서버들이 잃어버린 복제본을 다시 만드느라 복제 연결이 꽉 찹니다. 복제 연결이 따로 있어서 이때도 손님 요청과 스토리지 읽기는 영향을 받지 않습니다. 복제용 칸막이(REPL)는 방화벽과도 연결하지 않았습니다. Ceph끼리만 쓰는 길이라 밖으로 나갈 이유가 없기 때문입니다.

1-6 관리자가 장비에 접속하기까지

MANAGEMENT

엔지니어가 집에서 서버를 점검해야 합니다. 관리 접속은 정해진 한 길로만 들어옵니다.

#	구간	무슨 일이	왜
1	노트북 → VPN	2단계 인증(MFA) 후 관리 구역에 들어옴	관리망은 인터넷에서 직접 보이지 않음
2	VPN → 배스천	모든 관리 접속은 접속용 서버(배스천) 한 곳을 거침	누가 언제 무엇을 했는지 한곳에 기록
3	배스천 → 서버	방화벽이 정한 포트(SSH, 모니터링 등)만 열어 줌	관리 구역에서 서비스 구역으로는 필요한 것만
4	배스천 → 스위치	스위치는 관리망 스위치를 통해 접속. 개인 계정으로 로그인하고 명령을 모두 기록	서비스망이 통째로 멈춰도 관리망으로 들어가 고칠 수 있음

1-7 막혀야 하는 패킷들

BLOCKED

설계가 맞다면, 어떤 패킷은 목적지에 닿지 못해야 합니다. 구역을 나누고 방화벽을 둔 것이 실제로 무엇을 막는지 보겠습니다.

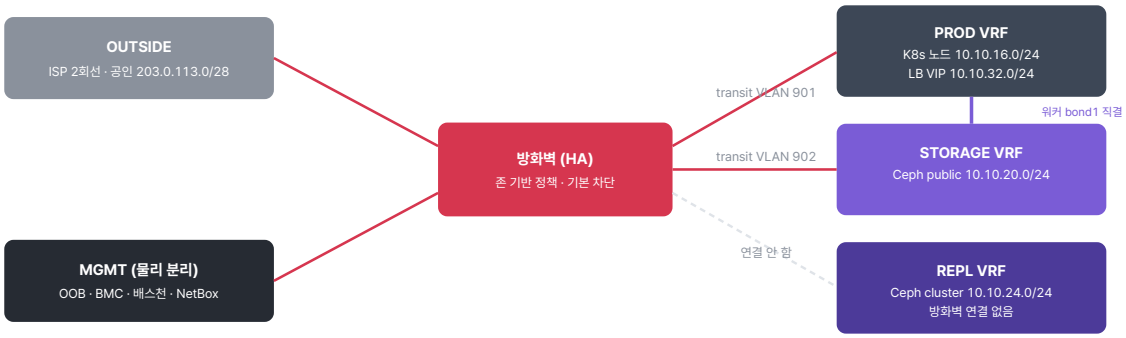


그림 4. 구역과 연결. 칸막이 사이를 오가는 길은 방화벽뿐이다

시도	어디서 막히나	왜
외부에서 서버 SSH(22번)로 접속	방화벽	열어 둔 목록에 없는 것은 전부 차단하고 기록
외부에서 스토리지로 직접 접근	방화벽	외부에서 스토리지·관리 구역으로 가는 허용 규칙이 없음
해킹당한 워커가 복제망으로	방화벽	서비스 칸막이에는 복제망으로 가는 길이 없고, 방화벽에도 그 길이 없음
설정 실수로 워커가 엉뚱한 주소를 알림	리프	리프는 워커에게서 서비스 주소만 받도록 걸러 냄

1-8

길이 끊어졌을 때

FAILURES

데이터가 오가는 도중에 장비가 고장 나면, 패킷은 이렇게 다른 길로 돕니다.

고장	패킷에게 일어나는 일	남는 영향
스파인 1대	리프가 끊김을 알아채고 남은 스파인으로만 보냄	속도는 절반, 서비스는 계속
리프 1대	서버는 남은 리프 쪽 연결로만 주고받음	그 랙 서버들의 속도가 절반
워커 1대	그 워커로 가던 손님 요청이 다른 워커로	남은 워커들이 나눠 받음
방화벽 1대, 회선 1개	대기 중인 방화벽과 다른 회선이 이어받음	수 초 ~ 수십 초 끊김
랙 하나 전체 정전	네트워크와 Kubernetes는 남은 랙으로 계속 동작	Ceph 4대가 2/1/1로 놓인 것이 약점. 복제본을 랙마다 하나씩 두어 데이터는 지키고, 다음 증설에서 2/2/2로 맞춤

PART 2

구축 계획

그 길을 실제로 까는 장비, 배선, 주소, 대역폭

2-1 장비와 배선

HARDWARE

핵심 장비(스파인, 방화벽)는 두 대씩 서로 다른 랙에, 리프는 랙마다 두 대씩 둡니다. 인터넷 연결은 별도 라우터 없이 방화벽이 직접 받습니다. 처음 트래픽(수 Gbps)에는 충분하고, 방화벽이 버거워지면 그때 라우터를 추가합니다.

장비	수량	사양	위치
스파인 스위치	**2**	100G 포트 32개	R1, R3
리프 스위치	**6**	25G 포트 48개 + 100G 포트 8개	랙마다 2대
방화벽	**2**	보안 기능을 켜고도 10Gbps 이상, 두 대 한 쌍	R1, R2
관리망 스위치	**3**	1G 포트 48개	랙마다 1대
콘솔 서버, 전원 장치	**1, 6**	비상 접속용 / 콘센트별 전력 측정	R2 / 랙마다 2개(이중 전원)

연결	속도·케이블	규칙	수량
리프 ↔ 스파인	100G 광케이블	리프 49번 포트는 spine1, 50번은 spine2로 고정	**12**
서버 ↔ 리프	25G 구리 케이블 (랙 안)	서버의 두 줄은 반드시 서로 다른 리프로	**42**
방화벽 ↔ 리프	10G	리프 47번 포트로 고정	**4**
방화벽 ↔ 방화벽	10G	두 방화벽이 상태를 나누는 선	**1**
인터넷 회선	10G	서로 다른 통신사 2개	**2**
관리 연결	1G / 10G	모든 장비의 관리 포트를 같은 랙 관리망 스위치로, 관리망 스위치끼리 연결	**26**

포트 번호를 규칙으로 고정해 두면, 현장에서 배선표를 보지 않아도 어느 포트가 무엇인지 알 수 있습니다. 케이블 87개의 배선표와 라벨은 설계 파일에서 자동으로 만듭니다.

2-2

서버·스토리지 연결

SERVER LINKS

서버마다 네트워크 카드(NIC) 두 장을 하나로 묶어(bond) 리프 두 대에 한 줄씩 꽂습니다. 한쪽 리프나 케이블이 끊겨도 다른 줄로 계속 통신하고, 평소에는 두 줄을 함께 써서 속도가 두 배입니다.

역할	첫 번째 묶음	두 번째 묶음
컨트롤러 3대	25G × 2: 서비스 + 스토리지 관리용	-
워커 5대	25G × 2: 서비스 (손님 요청)	25G × 2: 스토리지 읽기·쓰기 전용
Ceph 4대	25G × 2: 워커가 읽고 쓰는 쪽	25G × 2: Ceph끼리 복제 전용

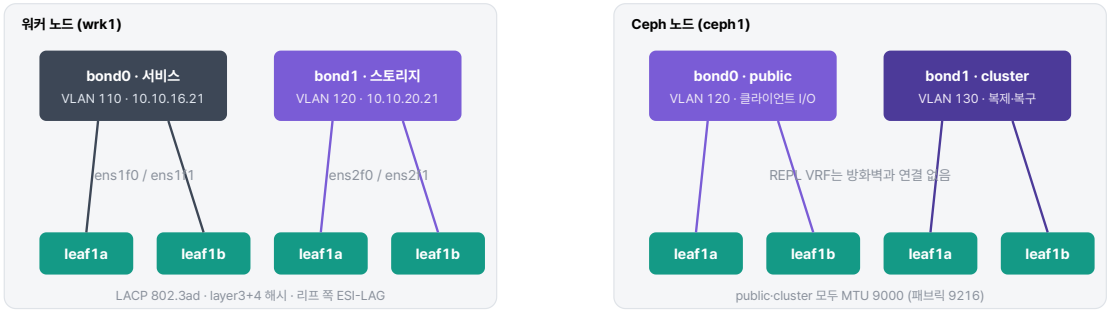


그림 5. 서버 연결. 같은 묶음의 두 줄은 반드시 다른 리프로 간다

Kubernetes와 Ceph 사이의 빠른 전송은 이 표의 두 번째 묶음이 맡습니다. 워커와 Ceph가 같은 스토리지 네트워크에 직접 연결되어 방화벽을 거치지 않으므로, 대용량 데이터가 방화벽 속도에 묶이지 않습니다.

2-3

내부·외부 분리와 방화벽

SEGMENTATION

내부 네트워크는 칸막이(VRF) 세 개로 나누고, 관리망은 스위치부터 따로 둡니다. 구역 사이를 오가는 길은 방화벽 하나이고, 기본은 모두 차단입니다. 방화벽 규칙은 파일로 관리해서, 사람이 보는 표와 실제 설정이 같은 내용에서 나오게 했습니다.

출발	도착	허용하는 것	결과
인터넷	웹 서비스 주소	웹(80, 443)만	허용
서비스 구역	인터넷	업데이트, 이름 조회, 시간 동기화	허용
관리 구역	서비스·스토리지 구역	원격 접속, 모니터링	허용
서비스 구역	스토리지 구역	전체	차단 (전용 연결로만)
스토리지 구역	인터넷	전체	차단
인터넷	스토리지·관리 구역	전체	차단
그 밖의 모든 트래픽			차단, 기록

2-4

IP 주소 할당과 관리

ADDRESSING

전체에 10.10.0.0/16을 쓰고, 용도마다 256개짜리 묶음(/24)을 하나씩 정해 둡니다. 서버 번호는 어느 네트워크에서나 같게 했습니다. wrk1은 서비스망에서도 스토리지망에서도 끝자리가 .21이라, 주소만 보고도 어떤 서버인지 알 수 있습니다.

주소 묶음	용도
`10.10.16.0/24`	서비스망
`10.10.20.0/24`	스토리지망
`10.10.24.0/24`	복제망
`10.10.32.0/24`	웹 서비스 대표 주소
`10.10.8~9.0/24`	관리망 (장비, 서버 관리 포트)
`10.10.0~1.0/24`	스위치 자체 주소
`10.10.64.0/18`	GPU 노드용으로 비워 둠

끝자리	누구
.1	게이트웨이
.11 ~ .19	컨트롤러
.21 ~ .29	워커
.31 ~ .39	Ceph
.100 ~ .199	앞으로 늘어날 서버

관리 방법. 주소는 설계 파일에서만 정합니다. 서버를 추가할 때는 번호 하나만 정하면 나머지 주소는 생성기가 계산하고, 중복되거나 범위를 벗어나면 설정을 만들기 전에 막습니다. 만들어진 주소 표는 자산 관리 도구(NetBox)에 넣어 누구나 조회할 수 있게 하고, 묶음이 70% 이상 차면 다음 묶음을 미리 잡아 둡니다.

2-5

대역폭과 QoS

BANDWIDTH · QOS

트래픽	얼마나	준비한 용량
손님 요청	처음에는 많을 때 2~5Gbps로 예상	인터넷 10G × 2, 방화벽 10G 이상
스토리지 읽기·쓰기	양이 많고 지연에 민감	서버당 25G × 2 전용
스토리지 복제	쓰기 1마다 복제 2, 장애 때 폭주	서버당 25G × 2 따로
리프 → 스파인	서버 쪽을 다 합쳐도 위쪽 연결의 1.18배	리프당 100G × 2, 70%가 넘으면 두 배로

QoS(혼잡할 때 누구에게 먼저 길을 내줄지 정하는 규칙). 평소에는 모두 남는 대역을 자유롭게 쓰고, 길이 막힐 때만 아래 비율을 최소한으로 보장합니다. 분류는 서버가 아니라 리프가 어느 네트워크에서 왔는지를 보고 합니다.

순서	대상	막힐 때 보장
1	스위치끼리 주고받는 제어 신호	항상 먼저
2	스토리지 (읽기·쓰기, 복제)	40%
3	웹 서비스	30%
4	Kubernetes 제어 신호	10%
5	그 밖의 것	나머지

패킷 크기도 구간마다 정했습니다. 스토리지는 한 번에 크게(9000바이트), 서비스는 인터넷과 같은 크기(1500바이트), 스위치 사이는 포장을 씌워도 들어가게 그보다 조금 크게(9216바이트) 맞췄습니다.

PART 3

운영 계획

그 길이 계속 잘 유지되게 지키는 방법

3-0

운영을 보는 관점

OVERVIEW

모니터링, 자동화, 장애 대응, 보안은 모두 앞에서 따라간 길을 지키기 위한 장치라고 봤습니다. 각 항목은 같은 구조를 직접 만들어 시험해 본 결과를 근거로 정했습니다.

항목	한마디로	쓰는 것
장애 대응	길이 끊기면 다른 길로 돈다	빠른 끊김 감지(BFD), 여러 길 나눠 쓰기(ECMP), 이중 연결
모니터링	데이터가 잘 흐르는지 지켜본다	Prometheus + Grafana
자동화	설정을 사람 손 없이 깐다	설계 파일 + 생성기 + Ansible
자산 관리	장비·케이블·주소의 지도를 갖는다	NetBox
보안	가면 안 되는 패킷은 막는다	방화벽, 구역 나누기, 접속 통제
확장	트래픽이 늘면 길을 넓힌다	미리 비워 둔 주소, 리프·스파인 추가
GPU	성격이 다른 트래픽에 새 길	GPU 전용 네트워크

3-1

가용성과 장애 대응

AVAILABILITY

고장이 났을 때 다른 길로 돌아가기까지 몇 초가 걸리는지가 핵심입니다. 같은 스파인-리프 구조를 직접 만들어 장애를 넣고, 0.2초마다 ping을 보내 끊긴 시간을 재 봤습니다.

케이블이 끊기는 고장은 **0.2초**면 다른 길로 돌았습니다. 더 위험한 것은 **스파인이 꺼지지 않고 응답만 멈추는 고장**이었습니다. 케이블은 살아 있어서 스위치들이 "상대가 죽었다"고 판단하기까지 9초를 기다렸고, 그동안 **7.6~8.8초** 동안 패킷이 사라졌습니다. 여기에 BFD(0.3초마다 서로 안부를 묻고, 세 번 연속 답이 없으면 끊긴 것으로 보는 기능)를 켜자 같은 고장이 **0.8~1.2초**로 줄었습니다. 그래서 이 설계는 스위치 사이 모든 연결에 BFD를 켭니다.

장치	하는 일
BFD	스위치 사이 연결이 끊긴 것을 1초 안에 알아챔
ECMP	스파인 두 대로 길을 나눠 쓰다가, 한쪽이 빠지면 남은 쪽으로만
서버 이중 연결	서버의 두 줄을 서로 다른 리프에 꽂아 리프 하나가 꺼져도 유지
방화벽 2대, 회선 2개	한쪽이 죽으면 다른 쪽이 이어받음

장애 대응 순서

원인보다 서비스를 먼저 살립니다. **알아챔 → 영향 확인 → 문제 부분 떼어 내기 → 복구 → 원인 분석** 순서로 움직이고, 예를 들어 에러를 내는 케이블은 원인을 찾기 전에 먼저 빼서 트래픽을 다른 줄로 돌립니다. 서비스가 멈춘 경우는 바로 호출하고, 이중화만 깨진 경우는 당일 안에 복구합니다. 같은 장애가 다시 나지 않게 막는 방법은 사람이 기억하는 대신 자동 점검 항목으로 남깁니다.

훈련. 시험 환경에 하루 5분짜리 장애 실습 도구를 만들었습니다. 날마다 고장 하나를 넣고 원인을 찾아 복구한 뒤 기록하는 방식입니다. 운영에서도 같은 방식으로 정기 훈련을 합니다.

3-2

모니터링

MONITORING

모니터링은 데이터가 잘 흐르는지 지켜보는 일입니다. Prometheus(수치를 모으는 도구)와 Grafana(화면으로 보여 주는 도구)를 씁니다. 같은 구조에 이 둘을 붙이고 장애를 넣어 봤더니, **화면은 정상인데 데이터는 막혀 있는 경우**가 있었습니다.

스위치 사이 연결 수를 설계 파일에서 계산해 두고 실제 수와 비교하는 알람은 잘 울렸습니다. 스파인을 멈추자 연결이 16개에서 8개로 줄며 알람이 5개 울렸습니다. 반면 패킷 크기 설정이 한 곳 어긋난 장애에서는 연결 수가 16개 그대로라 알람이 하나도 안 울렸는데, 큰 파일 전송은 멈춰 있었습니다. 큰 패킷이 버려진 횟수를 보는 알람을 추가하자 9초 만에 울렸습니다.

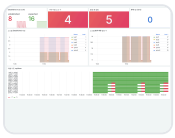


그림 6. 스파인을 멈췄을 때의 Grafana 화면. 연결 16개 중 8개, 알람 5개

그래서 네 가지를 봅니다

보는 것	어떻게	놓치면
스위치 연결 상태	연결 수를 설계 파일의 기대값과 비교	다른 길로 못 돎
포트와 대기열	사용률, 에러, 대기열에서 버려진 패킷	몰리는 구간을 못 봄
길 시험	작은 ping과 큰 ping(9000바이트)을 따로 보냄	큰 데이터만 막히는 장애를 못 봄
서비스	밖에서 웹사이트가 열리는지 확인	손님이 먼저 알게 됨

감시할 장비 목록도 설계 파일에서 자동으로 만들어, 새 장비가 감시에서 빠지는 일이 없게 했습니다.

3-3

자동화

AUTOMATION

자동화는 설정을 사람 손 없이 까는 일입니다. 사람은 설계 파일만 고치고, 설정 만들기·시험·배포·점검은 도구가 합니다. 장비에 직접 들어가 손으로 고치는 단계를 없애, 문서와 실제 설정이 어긋나지 않게 했습니다.

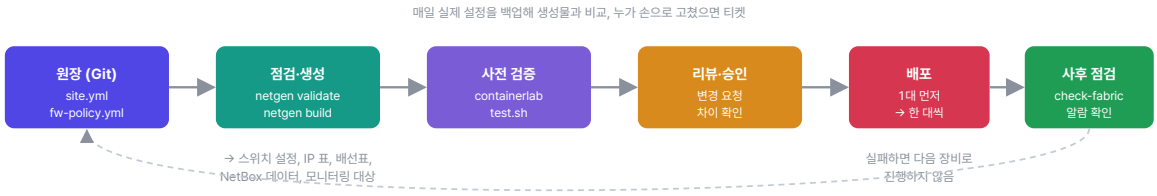


그림 7. 변경이 장비까지 가는 길. 장비에 직접 들어가 고치는 단계가 없다

단계	도구	막는 실수
설계 파일 수정	Git에 기록하고 다른 사람이 검토	누가 왜 바꿨는지 모르는 변경
설정 만들기	Python 생성기	주소 중복, 한쪽 리프에만 연결된 서버
미리 시험	같은 설정을 가상 환경(containerlab)에 띄워 시험	설정은 맞는데 통신이 안 되는 경우
배포	Ansible로 한 대씩, 실패하면 멈춤	실수가 전체 장비에 퍼짐
점검	배포 후와 매일 아침 연결 상태 자동 확인	누군가 손으로 고친 설정

시험 환경에서 배포 도구가 바뀔 줄만 미리 보여 주고, 바뀐 줄만 반영하고, 원래대로 되돌리고, 문법이 틀린 설정은 배포 전에 막는 것까지 확인했습니다. 점검 도구는 스파인 연결 하나를 끊자 끊긴 포트를 정확히 찾아냈습니다.

3-4

자산 관리

ASSETS

자산 관리는 장비, 케이블, 주소의 지도를 갖는 일입니다. **NetBox**를 제안합니다. 랙 배치, 장비, 포트, 케이블, IP를 한곳에 기록하는 오픈소스 도구이고, 다른 도구와 연결하기 쉽습니다. 이번 설계에서는 생성기가 설계 파일에서 NetBox에 넣을 데이터(장비, 케이블 87개, 주소)를 만들어 줍니다.

기록	내용	어디서 오나
하드웨어	랙 위치, 모델, 시리얼, 자산번호, 보증 만료일	설계 파일 + 입고 때 등록
연결	양쪽 포트, 케이블 번호와 종류	생성기가 만든 배선표
IP	주소 묶음, 누가 어떤 주소를 쓰는지	생성기가 만든 주소 표

기록과 실제를 맞추는 방법. 스위치는 자기 옆에 누가 꽂혀 있는지 알고 있습니다(LLDP). 이 정보를 매일 모아 NetBox의 케이블 기록과 비교하고, 다르면 알려 줍니다. 잘못 꽂힌 케이블을 서비스에 투입하기 전에 잡을 수 있습니다. 같은 방식으로 실제로 쓰이는 주소 중 기록에 없는 것을 찾아, 몰래 붙은 장비가 없는지도 확인합니다.

3-5

보안 강화

SECURITY

보안은 1-7에서 본 막힘을 계속 유지하는 일입니다. 패킷을 막는 곳은 세 군데입니다.

어디서	무엇을
방화벽	기본은 모두 차단. 밖에서는 웹(80, 443)만 열고, 웹 공격을 걸러 내는 기능을 켬. 대량 공격(DDoS)은 통신사 방어 서비스로
네트워크 칸막이	서비스·스토리지·복제를 칸막이로 나눠, 하나가 뚫려도 다른 칸으로 가는 길이 없게. 리프는 서버가 알리는 주소 중 허용된 것만 받음
관리 접속	VPN(2단계 인증) → 접속용 서버 → 관리망. 개인 계정으로만 로그인하고 모든 명령 기록

라우팅 쪽 보안은 작은 인터넷 환경을 따로 만들어 시험해 봤습니다. 남의 주소를 가로채려는 잘못된 경로 알림을 입구의 필터가 전부 막는 것을 확인했고, 이를 바탕으로 리프에도 같은 필터를 넣었습니다.

정책을 계속 맞게 유지하는 방법. 방화벽 규칙은 파일로만 바꾸고 다른 사람이 검토합니다. 분기마다 한 번도 쓰이지 않은 규칙과 기록에 없는 장비를 정리하고, 장비 보안 업데이트는 정기 점검 때 반영합니다.

3-6

확장 계획

GROWTH

트래픽이 늘면 길을 넓히되, 길의 모양은 바꾸지 않습니다. 랙을 하나 더하면 리프 2대를 기존 스파인에 꽂으면 끝이고, 데이터는 여전히 리프 → 스파인 → 리프 세 칸으로 다닙니다. 스파인 포트는 32개 중 6개만 쓰고 있어 랙 8개까지는 이 모양 그대로 늘어납니다.

단계	언제 (알람)	무엇을
1. 서버 추가	리프 포트 80% 사용	설계 파일에 한 줄. 주소는 비워 둔 번호에서
2. 연결 늘리기	리프-스파인 연결 70% 사용	리프와 스파인 사이를 두 줄로
3. 랙 추가	랙 전력 80% 사용	리프 2대만 더함
4. 스파인 추가	스파인 포트 부족	스파인을 3~4대로 늘림

이렇게 쉽게 늘리려면 두 가지가 미리 돼 있어야 한다는 것을 시험 환경에서 배웠습니다.

- **스위치끼리 연결에는 주소를 붙이지 않습니다(BGP unnumbered).** 리프를 더할 때마다 연결용 주소를 계산할 필요가 없어집니다.
- **여러 길에 고르게 나눠 보내게 합니다.** 시험해 보니 기본 설정에서는 트래픽이 스파인 한쪽으로 크게 쏠렸고(3:42), 포트 번호까지 보도록 바꾸자 고르게 나뉘었습니다(21:23). 고르게 안 나뉘면 스파인을 늘려도 소용이 없습니다.

3-7

GPU 노드 추가

GPU

AI 학습에서는 GPU끼리 계산 결과를 끊임없이 주고받습니다. 한꺼번에 아주 많은 양이 오가고, 패킷 하나만 잃어도 전체 학습이 기다립니다. 지금까지의 트래픽과 성격이 완전히 다르므로, 같은 원칙대로 **GPU 전용 네트워크를 하나 더 둡니다.**

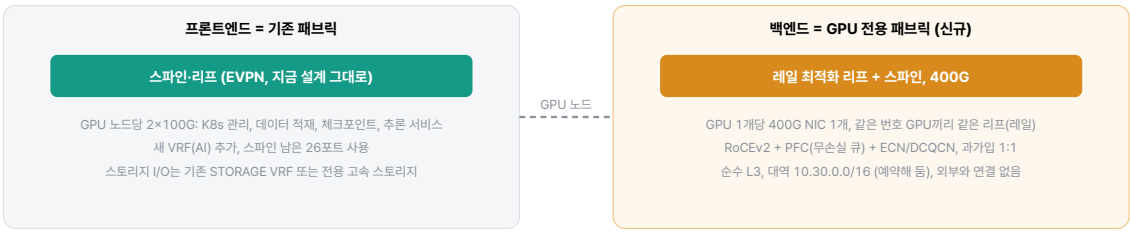


그림 8. GPU를 추가할 때의 구조. 기존 네트워크는 그대로 두고 GPU끼리만 다니는 네트워크를 따로 둔다

고려할 점	내용
전용 네트워크	GPU끼리만 다니는 빠른 네트워크(400G)를 따로 두고, 기존 네트워크는 관리와 데이터 공급, 추론 서비스에만 씁니다.
패킷 잃지 않기	GPU 통신(RoCE)은 패킷 손실에 약합니다. 길이 막히기 전에 보내는 쪽 속도를 미리 줄이게(ECN) 하고, 정말 꽉 차면 잠깐 멈추게(PFC) 합니다. 작은 시험 환경에서 이 방식으로 손실이 약 50%에서 0으로, 지연이 19.8ms에서 1.3ms로 줄었습니다.
전력·냉각	GPU 서버 한 대가 10kW 안팎이라 지금 랙(8kW)에 들어가지 않습니다. 전력이 큰 전용 랙이 필요하고, 네트워크보다 큰 비용입니다.
스토리지	학습 데이터를 아주 빠르게 읽어야 해서 GPU 서버에 빠른 디스크(NVMe)를 두고, Ceph는 원본 보관에 씁니다.
추론만 한다면	GPU끼리 주고받을 일이 적어 별도 네트워크 없이 기존 네트워크로 시작할 수 있습니다.

3-8

랙 구성과 비용

RACKS · COST

랙	네트워크 장비	서버	예상 전력
R1	spine1, 리프 2, 관리망 스위치, fw1	ctrl1, wrk1, wrk2, ceph1, ceph2	**6.1 kW**
R2	리프 2, 관리망 스위치, fw2	ctrl2, wrk3, wrk4, ceph3	**4.9 kW**
R3	spine2, 리프 2, 관리망 스위치	ctrl3, wrk5, ceph4	**4.2 kW**

고려사항	설계에서 한 일
성능	트래픽마다 전용 연결, 스토리지는 큰 패킷, 위쪽 연결 여유 있게
확장성	스파인 포트 6/32만 사용, 랙 추가 시 리프 2대만
가용성	데이터가 지나는 모든 장비 이중화, 핵심 장비는 다른 랙, 1초 안에 우회
보안	구역 나누기, 관리망 분리, 방화벽 기본 차단
유지보수	설계 파일 하나에서 설정 생성, 한 대씩 배포, 포트 번호 고정
비용	별도 라우터 생략, 연결은 필요할 때 늘림, 랙 안은 저렴한 구리 케이블, 무료 도구

	1안: 기술지원 받는 브랜드 장비	2안: 저렴한 범용 장비
네트워크 (스위치, 방화벽, 케이블, 랙)	**약 2.6 ~ 4.6억**	**약 1.8 ~ 3.3억**
서버 12대		**약 3 ~ 5억**
처음 구축 합계	**약 5.6 ~ 9.6억**	**약 4.8 ~ 8.3억**

처음에는 1안을 추천합니다. 문제가 생겼을 때 제조사 지원을 받을 수 있어 운영을 안정시키기 좋습니다. GPU 네트워크처럼 포트가 크게 늘어나는 단계에서 2안을 검토합니다. 표준 기술만 썼기 때문에 두 장비를 섞어 써도 운영 방식은 같습니다. 매달 드는 비용으로는 인터넷 전용회선 2개, 전기와 냉방(약 15kW), 장비값의 연 10~20% 정도 유지보수비가 있습니다. 금액은 공개된 시장 가격으로 추정한 것이며 견적은 받지 않았습니다.

시험과 한계

LIMITS

PART 1의 길들은 설계 파일에서 만든 설정을 가상 환경(containerlab)에 그대로 올려 시험했습니다. 리프 두 대가 한 서버를 함께 받는 묶음의 실제 전환, 방화벽 두 대의 전환, QoS의 실제 효과는 가상 환경에서 재현하기 어려워 실제 장비를 들일 때 시험할 항목으로 남깁니다.